

INTERVIEW PREP · DAILY NOTES

Python & SQL Working Session

Functional programming with map/filter/lambda, sorting with custom keys, and SQL subqueries — including correlated subqueries and Nth-highest salary patterns.

map() & filter()

lambda expressions

sorted() with key

SQL subqueries

correlated subqueries

table aliases

Python — Functional Tools

PYTHON

1 map() — Transform Every Element

`map()` is used when we want to transform every element of a collection.

```
salaries = [45000, 55000, 65000, 75000, 85000]

salaries = list(map(lambda a: a + a * 0.10, salaries))

print(salaries)
```

This increases every salary by 10%.

PATTERN

```
list(map(lambda x: transformation, data))
```

THINK

map = change / transform every element

2 filter() — Select Elements by Condition

`filter()` is used when we want to keep only elements satisfying a condition.

```
salaries = [45000, 55000, 65000, 75000, 85000]

salaries = list(filter(lambda a: a > 60000, salaries))

print(salaries)
```

OUTPUT

```
[65000, 75000, 85000]
```

THINK

filter = select elements

3 Combining filter() + map()

We can filter first, then transform.

```
salaries = [45000, 55000, 65000, 75000, 85000]

salaries = list(filter(lambda a: a > 60000, salaries))
salaries = list(map(lambda a: a + a * 0.10, salaries))
```

```
Original salaries
|
filter > 60000
|
65000, 75000, 85000
|
map +10%
|
71500, 82500, 93500
```

INTERVIEW EXPLANATION

filter() selects the required elements, while map() transforms the selected elements.

4 Filtering Dictionaries

For a list of employee dictionaries:

```
employees = [
    {"name": "Alice", "salary": 80000, "department": "IT"},
    {"name": "Bob", "salary": 60000, "department": "HR"}
]

employees = list(
    filter(lambda employee: employee["salary"] > 70000, employees)
)
```

Here, `employee["salary"]` accesses the salary of the current employee dictionary.

5 Filtering with Multiple Conditions

Instead of chaining two filters:

```
employees_it = list(
    filter(lambda employee: employee["department"] == "IT", employees)
)

employees = list(
    filter(lambda employee: employee["salary"] > 70000, employees_it)
)
```

...we can combine both conditions into one filter call:

```
employees = list(
    filter(
        lambda employee:
            employee["department"] == "IT"
            and employee["salary"] > 70000,
        employees
    )
)
```

INTERVIEW ANSWER

"I could use two filters, but since both conditions are required for the same selection, I can combine them using `and` in one lambda."

6 map() to Extract Names

```
employees_names = list(
    map(lambda employee: employee["name"], employees)
)
```

```
employee dictionary
|
employee["name"]
|
employee name
```

Example: `[{"name": "Alice", ...}, {"name": "Bob", ...}]` becomes `["Alice", "Bob"]`

7 Filtering + Mapping (Common Interview Pattern)

```
employees_names = list(
    map(
        lambda employee: employee["name"],
        filter(
            lambda employee:
                employee["department"] == "IT"
                and employee["salary"] > 70000,
            employees
        )
    )
)
```

```
employees
|
filter
|
IT + salary > 70000
|
map
|
extract names
```

8 Transforming the Full Dictionary

Question: What if I want the full dictionary after increasing salary?

This returns *only* salaries, because the lambda returns only the salary:

```
employees = list(
    map(lambda employee: employee["salary"] * 1.10, employees)
)
```

To keep the whole dictionary, spread the existing fields and update salary:

```
employees = list(  
    map(  
        lambda employee: {  
            **employee,  
            "salary": employee["salary"] * 1.10  
        },  
        employees  
    )  
)
```

NOTE

The `**employee` keeps the existing dictionary fields and updates salary.

9 sorted() + lambda

Attempted (incorrect):

```
employees = list(sorted(lambda employee: employee["salary"], employees))
```

Correct syntax:

```
employees = sorted(  
    employees,  
    key=lambda employee: employee["salary"]  
)
```

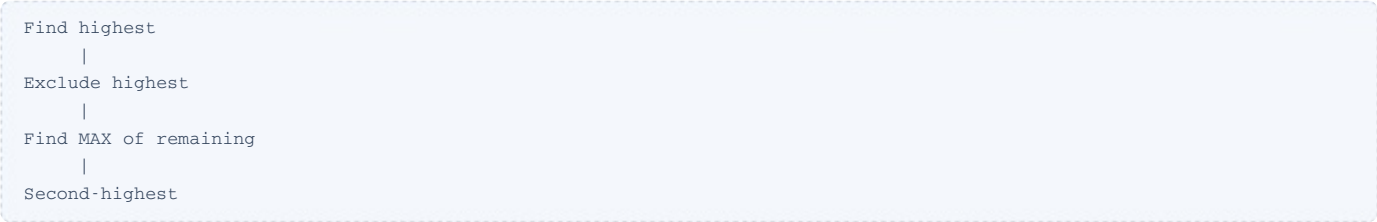
Descending order:

```
employees = sorted(  
    employees,  
    key=lambda employee: employee["salary"],  
    reverse=True  
)
```

```
sorted()  
|  
key = what should I sort by?  
|  
lambda employee["salary"]
```

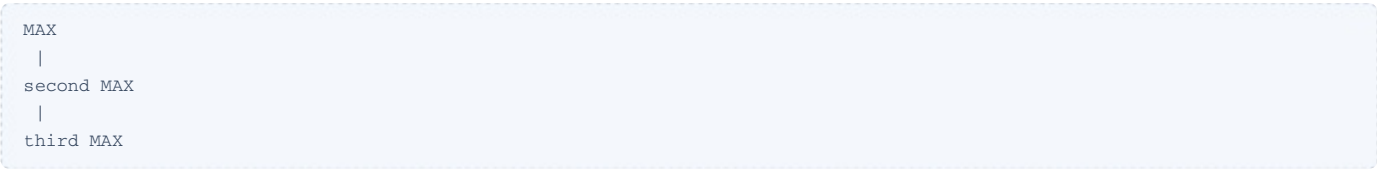
10 Second-Highest Salary

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
);
```



11 Third-Highest Salary

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
    WHERE salary < (
        SELECT MAX(salary)
        FROM Employee
    )
);
```



12 Second-Highest Salary in a Department

For department 1:

```
SELECT MAX(salary)
FROM Employee_1
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee_1
    WHERE departmentId = 1
)
AND departmentId = 1;
```

CLAUSE	PURPOSE
Inner — WHERE departmentId = 1	Find the highest salary of department 1
Outer — AND departmentId = 1	Find the second-highest salary only among department 1 employees

13 Second-Highest Salary for Every Department

```
SELECT e.departmentId,
    MAX(e.salary)
FROM Employee_1 e
WHERE e.salary < (
    SELECT MAX(e2.salary)
    FROM Employee_1 e2
    WHERE e2.departmentId = e.departmentId
)
GROUP BY e.departmentId;
```

KEY CONCEPT

e2.departmentId = e.departmentId — this is a **correlated subquery**. The inner query uses the department from the current outer row.

14 Employees Above Their Department Average

```

SELECT e.name,
       e.salary,
       e.departmentId
FROM Employee_1 e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee_1 e2
    WHERE e2.departmentId = e.departmentId
);

```

```

Employee salary
|
Find employee's department
|
Calculate department average
|
salary > average?
|
YES -> return employee

```

15 SQL Table Aliases

`FROM Employee e` means: `Employee` is the actual table, `e` is the alias. So `e.salary` means "salary from the Employee table reference named e."

e vs e2: when the same table is referenced twice, we use different aliases — `e` for the outer Employee, `e2` for the inner Employee. So `e.salary` and `e2.salary` refer to different references of the same table.

IMPORTANT

`e`, `e2`, `e3` have no special SQL meaning — they are simply aliases chosen by us. You could equally use `Employee outer_employee` and `Employee inner_employee` instead.

Today's Most Important Takeaways

SUMMARY

Python Building Blocks	Mental Models
<code>map()</code> transform	<code>map</code> "Change this"
<code>filter()</code> select / filter	<code>filter</code> "Keep this"
<code>lambda</code> small anonymous function	<code>sorted</code> "Arrange this"
<code>sorted()</code> sort using key	<code>WHERE</code> filter rows
	<code>GROUP BY</code> create groups
	<code>HAVING</code> filter groups
SQL Building Blocks	Table Reference Roles
<code>Subquery</code> query inside another query	<code>e</code> outer table reference
<code>Correlated</code> inner depends on outer	<code>e2</code> inner table reference
<code>e</code> alias for outer table reference	
<code>e2</code> alias for inner table reference	